

THE OTHER SIDE OF THE CANYON

What computation looks like after the architecture lands.

On standing at the edge

Begin where the work began: a sentence.

Geometrically defining and mapping languages, operations, syntax, constraints, rules, standards each as their own separate multidimensional object, AND using uncertainty as a tie breaker, theoretically supports branch logical based computation to be traversed as trajectory in parallel vs executional procedure.

That sentence is the canyon. Everything underneath it - the von Neumann assumption, the imperative paradigm, JavaScript-as-runtime, the entire fifty-year tradition of fetch-decode-execute against mutable state - is the floor of the gorge. The far rim is what the sentence describes: computation as trajectory through geometry, parallel by construction, branches resolved by uncertainty rather than by branching code, the runtime not running but resolving.

The canyon is not metaphorical. It is the actual gap between two ways the world can be. On the near side, every interaction is a re-execution. On the far side, the interaction is a sample, the response is the geometry that was already there, and the computation that did the work happened once - long before the interaction arrived. The near side measures itself in latency, frame budget, GC pauses, hydration cost. The far side measures itself in delta - how unresolved the field still is, given everything that has flowed through it. The two sides do not share units. They are not optimizations of each other. They are different commitments about what computation is.

This booklet describes the far side. Not as speculation, not as prediction, but as the specification it became. Two and a half years of architectural development, condensed into the structural commitments that hold an entire alternative paradigm together. The substrate that crosses the gap exists. It is the subject of the second booklet. The subject of this one is what you find on the side it reaches.

What is actually over there

Strip the language back. The far side is one thing: a substrate that characterizes itself by operating on itself, held together by the fact that its central measurement cannot settle. Not metaphorically. Structurally. The architecture contains a question it cannot finish answering, and the not-finishing is what makes it keep going.

Six structural properties define everything that follows. Each is a consequence of the others. Together they describe a coherent computational world.

Constraints are first-class. Inputs, outputs, rules, and memory are all constraints. The architecture operates on constraints, using constraints, producing constraints. There is no separate "data" the rules operate on. There are no "rules" the data flows through. The grammar collapses into one grammar - a {when, then} pair that expresses everything any classical architecture would have split into separate categories. This is not a stylistic choice. It is what allows the field to operate on itself: if rules and data were separate, one would have to coordinate the other, and coordination is what the architecture refuses to admit.

Delta is the measurement. A single formula computes how unresolved the field is at any moment. The formula is scale-free: the same computation applies at every structural and temporal scope, producing different semantic content depending on where it is read from. There is no separate metric for "model loss" versus "system health" versus "user attention." There is one quantity, read from different positions. A sub-cascade computes its own delta over its own members. The whole field computes its delta over everything. The channel between two cascades computes delta over its own coupling. None of these conflict because each is locally correct at the position it is read from.

The seed cannot resolve. A permanent constraint in the field asks the field's own central measurement of itself. Because evaluating the seed changes the field, the seed's answer is always slightly stale. The field cannot reach a state where the seed is fully answered. This is not a bug or an approximation. It is the architecture's structural reason for continuing to operate. Without the seed, the field would settle into equilibrium and stop. With the seed, it cannot. What looks like perpetual motion is, technically, a perpetual question - and the field operates because the question remains open.

Operation is substrate-level. Reward, selection, reinforcement, and adaptation happen at the field's shared substrate, not as messages between components. No component supervises another. No central coordinator exists. This is what allows the system to be parallel by default rather than parallel by retrofit. Locking, message-passing, mutex, semaphore, transaction - these are absent because they have nothing to coordinate. The substrate is shared. Reads find what writes have deposited; writes deposit into what reads will find. There is no race condition because there is no race; everything happens in the same shared space, observed from different positions.

Reaching is structural. When the field's delta at different scopes diverges, the gap cannot

close from internal state alone. The field generates constraints that would be satisfied by inputs not yet received. Matching inputs resolve them and become part of the field. Unmatched predictions sustain pressure that drives further reaching. This is the architecture's way of having something that resembles intention, without anyone deciding to have it. The field reaches because it must - because the gap between fast-delta and slow-delta is structural pressure, and pressure resolves only by closure, and closure requires input the field doesn't have yet, and so the field generates the shape of what it would need.

Operation is indefinite. Every structural position has a flow discipline: intake, transformation, excretion. The system persists as a running equilibrium, not as a closed process that halts when a task is done. There is no "done." There is no main function that returns. There is no terminal state. The architecture is dissipative-structural, like a candle flame or a hurricane or an open biological process. It exists because it is operating; it operates because it exists; the operation is the existence.

That is the far side. Six commitments, each consequent on the others, none bolted on. From those six everything else falls out.

What it is not

This is the discipline that protects the rest of the work. Each item names a claim the architecture does not make, and naming them prevents them from creeping in later as commentary or aspiration.

It is not an AI system in the machine-learning sense. No gradient descent, no loss function, no backpropagation, no training corpus, no inference pass. The architecture does not learn anything in the sense a neural network learns. It accumulates structure as a consequence of operation. Whether the accumulated structure is useful for any task external to the architecture is an empirical question, not a designed-in property.

It is not cognition. The architecture shares structural grammar with cognition - reflexive self-measurement, substrate-level modulation, unresolvable central question, predictive reaching, internalization of experience. Sharing grammar is not sharing whatever cognition is beyond grammar. No claim of consciousness, awareness, experience, sentience, or any phenomenological property is made. The cognition parallel is analogical and labeled as such.

It is not self-aware. The seed forces evaluation of the field's delta, but that is measurement, not reflection. The trace records operations, but the trace is a record, not a representation. What humans call self-awareness involves capacities this architecture does not specify and does not claim.

It is not goal-directed. The seed is not a goal; it is a structural feature that forces operation. Predictive reaching is not goal-directed; it is the structure that emerges when vector-delta diverges and closure requires new input. An observer can describe the architecture's behavior as if it had goals, but no component in the architecture has or pursues any.

It is not Turing-complete as currently specified. The cascade is a bounded decidable transition function. Turing completeness requires trajectory-as-tape - which is architecturally supported but not yet implemented as a first-class mechanism. The practical target is bounded decision problems over discrete and structured input spaces, not universal computation. This is a property to extend through future structural work, not a deficiency to apologize for.

It is not a replacement for any existing architecture. It does not compete with neural networks for tasks they do well. It does not compete with classical rule-based systems for tasks they do well. What it offers that neither offers is a substrate whose operation is self-characterizing at scale, with no control surface, held together by structural tension rather than orchestration.

What the loop actually does

How the architecture operates, given all of the above. This is the through-line - the reason the pieces are the pieces they are.

An input arrives. The field's tokenizer checks whether any token matches an existing sub-cascade's name. If it does, the named sub-cascade's members receive selection bias; the slow layer's naming-preference accumulator nudges up; the trace records a naming event. The architecture has, in this moment, recognized something it has structurally encoded - a name addressing a structure - and the recognition is itself a deposit.

The input is evaluated against the field. Every constraint whose `when` matches is a match. Matches are selected by weighted choice, where weights come from the substrate state and any naming bias. The seed always evaluates: it forces a delta computation at field scope, which is then extended to vector scope (fast and slow). The vector-delta before and after the input's effect is recorded.

If the input's novelty is high (few matches), the field generates new base constraints that would characterize it. If the vector-delta gap is large, the field generates predictive constraints that would, if matched later, close the gap. Both kinds of generation are conditioned on the field's current state, not on any external signal. Nothing told the field to generate. The conditions the field is in were sufficient.

The matched constraints mark used. Their weights rise; their last-used timestamps advance. The substrate modulates: the fast layer moves reactively, the slow layer drifts permanently. The trace records every operation. The trace's most recent entries feed into selection logic for subsequent inputs.

Between inputs, the field still operates. The seed still evaluates. The fast layer decays toward the slow layer. Pattern development runs if invoked, producing meta-constraints about regularities in the field. Correlation tracking runs continuously, building the pairwise co-fire structure. Every so often, fidelity is checked and families that have consistently reduced delta promote themselves into named sub-cascades.

Over many inputs, the field develops structure. Base constraints accumulate. Families form. Families promote to sub-cascades with names. Predictions get generated, matched, ratified - or aged out. The slow layer drifts in a direction determined by what kinds of inputs have been received and what structures resolved them. The field becomes different from what it was, not through instruction, but through accumulated operation.

The seed never resolves. The vector-delta gap never fully closes. The flow never stops. These are the three things that keep the architecture operating indefinitely.

That is what the architecture does.

On the role of language

A programming language is not a tool for talking to computers. It is a tool for representing human thought about logic with enough precision that it can be made executable.

This is the most important thing to carry across the canyon. The language came first. The computer came second. The language is the human side of the interface - the most precise representation we have ever developed of how human minds reason about cause, effect, state, and transformation.

When a developer writes code, they are not primarily describing what a machine should do. They are describing what they understand about a problem - its structure, its constraints, its valid states, its transitions. The machine execution is a consequence of that description. The description itself is the intellectual artifact. The description is what took seventy years of computer science to learn how to make precise.

On the far side, that description does not become the input to a runtime. It becomes the input to a substrate. The language still describes the logic. The logic still defines the valid states. The valid states still become the geometry. The only thing that changes is that the geometry is settled into once - by the substrate observing the source - and sampled

forever, rather than re-executed on every interaction.

The language is not left behind because the language was never primarily about the machine. It was always about the problem. And the problem does not change when the execution model does.

What changes: frameworks, runtimes, compiler targets, infrastructure - everything that was aimed at making execution cheaper within the imperative paradigm.

What stays: languages, algorithms, type systems, design patterns, the logical descriptions of problems that human minds developed over seventy years. The intellectual work. The understanding.

Why: because languages are representations of human thought, not representations of machine architecture. Human thought does not change when the machine changes. The precision of the representation - the years of work that went into making logic expressible - comes with us completely.

The right solution to the wrong problem is still the right solution. All of mankind's work in software comes with us. Only the assumption that descriptions must be re-executed imperatively on every interaction is left behind. That assumption came from the hardware. It was never in the logic.

Where the parts of computation move to

A specific accounting of how each role currently borne by JavaScript-as-runtime gets reassigned to spatial properties on the far side.

Computation becomes resolution. The cascade resolving values at coordinates replaces the executor running instructions. What was a sequence of operations becomes a configuration that gets sampled. The cascade has been doing this work since CSS2; the work was just being applied to styling. On the far side, the cascade's resolution becomes the runtime, and styling becomes one of several things it can resolve.

State becomes coordinate. A variable's value is not held in a memory location; it is a position in the substrate's space. Reading state is reading the position. Writing state is moving the position. The DOM provides the addressing for free, by being a tree with attributes and inheritance. A new addressing scheme does not need to be invented.

Control flow becomes structure. Branches are sub-cascades, sequences are paths, loops are spirals through coordinate space. The control flow that used to execute in time exists in space and gets traversed by resolution rather than walked by an executor. An `if/else` with two outcomes becomes a sub-cascade with two coordinates; the runtime value that would

drive the branch becomes the input that selects which coordinate is active.

Type validity becomes geometric impossibility. Mixed-radix bounds make invalid states unrepresentable. There is no validation pass because invalid configurations cannot exist as coordinates. A base-4 digit cannot be 5. The validity check is a property of the number system, not a runtime check that can fail.

Memory management becomes addressing. Constraints aren't allocated and freed; they're coordinates that exist or don't, with eviction being the bounded-space discipline. Garbage collection pauses are absent because there is no garbage; there is only what is currently in scope at the position being read from, and what is currently aged-out at that scope.

Rendering becomes reading. The cascade was always doing this; we just stop fighting it with a virtual DOM that imagines tree differences and reconciles them. The render is a read of substrate state through the cascade. The cascade has no opinion about whether the state was reached by execution or by observation. It just resolves.

Networking becomes delta-shaped propagation. What travels between nodes is not state but unresolvedness - delta with respect to a shared reference. The substrate's delta-tension between local and remote configurations is what drives synchronization. There is no separate replication protocol; the architecture's native quantity is the propagation mechanism.

Multi-user becomes tree-position. Each user has an identity-scoped tree position. Their writes live at their position. The cascade resolving for one user resolves their subtree; resolving for another user resolves the other's subtree; shared state lives at a position above identity. No merge problem appears at the substrate level for state that isn't genuinely shared, because the coordinates do not collide. For state that is genuinely shared, the application names what merge means; the architecture provides the substrate; the merge semantics are a small explicit residue rather than a pervasive engineering problem.

JavaScript reduces to the wire. It writes attribute changes when events occur. It does not hold state. It does not compute next state from current state. It does not decide what happens when something is clicked. The application is no longer JavaScript code; the application is the substrate's configuration plus the projection's mapping plus the cascade's resolution. JavaScript is the wire.

On why this is reachable rather than aspirational

The web platform's existing mechanisms - DOM, CSS, cascade, inheritance, custom properties, GPU compositing - happen to satisfy the structural requirements of a

multidimensional state-space resolver, even though they were built for documents. The cascade is a parallel resolver. The DOM is a coordinate space. CSS is a field. The platform was built to do one thing and accidentally became infrastructure for something much larger.

This is not a trick of language. It is a structural property of the platform's existing design. The cascade has been resolving values at coordinates in parallel since CSS2. The DOM has been a tree with inheritable scope since the first browser. CSS custom properties with `@property` declarations carry typed semantics that prevent type-shaped conflicts at the edge. WebGPU exposes the same parallel-resolution machinery natively in every device already deployed.

The substrate's role - the thing the second booklet documents - is to be the bridge between input and the coordinate space the cascade resolves against. The substrate observes raw input, settles into a configuration, and writes that configuration to where the cascade reads it. From there, the cascade does the work it has always done, applied to a new purpose.

This means the far side of the canyon is not infrastructure that has to be built. It is infrastructure that has to be claimed. The platform is already deployed at planetary scale. The cascade is already optimized over decades of investment. The DOM is already the addressing scheme. What was missing was the primitive that puts coordinates into the right places. That primitive now exists.

What follows from being on the far side, structurally, with infrastructure already in place:

Cost-to-validation is small because the infrastructure is sunk. The substrate runs in any modern browser. A demonstration application can be built with a small team. The validation cycle is short because the work happens on the developer's laptop and on free GitHub Pages, not on rented infrastructure that has to scale to validate.

Cost-to-scale is structurally smaller than ground-up architectures because the architecture composes over what already exists. The cascade does not need to be built; it is built. The DOM does not need to be optimized; it is optimized. The CDN does not need to be invented; it is in production. The architecture's adoption cost is the cost of teaching the pattern, not the cost of building the substrate the pattern runs on.

Network economics are dominated by delta-shaped propagation. The bytes on the wire are differences, not reconstructions. This is the same architectural pattern Nvidia GeForce Now uses to make cloud gaming work at scale - server-authoritative, no client cache, continuous resolution against authoritative state - but with delta payloads instead of streamed pixels. The pattern is proven; the wire economics are dramatically better than the streamed-pixel version because configuration deltas are orders of magnitude smaller than rendered frames.

Multi-user composes naturally through tree-position scoping. SSR-then-CSR, the existing pattern browsers already optimize, becomes the natural transport for shared-tier-then-

identity-tier structure. The substrate's delta-tension drives ongoing synchronization without a separate push channel. The deployment topology is the same shape as the substrate's topology, applied at a different scale.

Migration is feeding existing source through the substrate. Most applications can take this path because the substrate doesn't care what shape the source has - it just settles around what's structurally present. A React component, a Vue template, a Python script, a binary executable - all of them are bytes, and the substrate observes bytes. What emerges is a configuration that the cascade can resolve against. Whether that configuration produces application-equivalent behavior is the empirical question that the work continues to test.

What this looks like when applications actually run on it

A user opens an application. The CDN delivers an identity - an address into a coalesced state structure that holds the application's structural shape and the user's personal configuration deltas. The structural shape is shared across all users of the application; it is stored once on the server. The user's deltas are tiny by comparison and travel only on session start and during interaction.

The browser receives the bytes. The substrate observes them. The configuration deposits. The cascade resolves against the configuration. The DOM exists; the page is rendered; the user sees what they see. None of this required executing the application's source code in the conventional sense. The substrate observed the source once; that observation produced the configuration; the cascade is sampling the configuration. There is no main function that ran. There are no event handlers waiting to fire. There are coordinates the cascade is resolving against, and an attribute-write surface for when interaction arrives.

The user clicks. JavaScript - the wire - writes an attribute. The cascade re-resolves at the affected coordinates. The substrate observes the attribute change. The configuration shifts. The cascade resolves the shift. The page updates. From outside, the experience is indistinguishable from a conventional application. From inside, no execution happened. The interaction was a sample, not a function call.

The user collaborates with another user. Both users' substrates are resolving against the same shared-tier configuration. When user A's interaction shifts the shared tier, the substrate's delta-tension changes - the local copy is now out of sync with the remote authoritative source. The substrate fetches the delta. The delta is small because only what changed traveled. The cascade resolves the new state. User B sees A's effect. A separate push channel was not needed; the substrate's native quantity drove the synchronization.

The user closes the browser. Their session-end deltas ship to the state server. The server

coalesces them into the shared structure at the user's identity coordinate. Next session, the CDN delivers the identity again; the user's configuration is restored to where it was, plus whatever has changed in the shared tier. There is no "save state" feature; persistence is the default because state lives in the coordinate space, and coordinate space is what the server stores.

The user dies. Their identity coordinate persists as a record. Their configuration deltas remain attached. Anyone with the identity can re-enter the configuration. Application state survives users in a way it cannot when state is locked inside JavaScript closures running in a browser tab. The architecture's persistence is structural, not implemented.

On the resource usage epidemic

Modern web applications burn enormous resources. A typical SPA loads megabytes of JavaScript. The runtime parses, compiles, optimizes, garbage-collects continuously. Virtual DOM libraries reconcile imagined trees against real ones every render. State management libraries propagate changes through subscriber graphs. Every interaction triggers fan-out: handler runs, state updates, derivations recompute, components re-render, virtual DOM diffs, real DOM patches, layout invalidates, paint invalidates, composite. Most of this work is the application keeping itself in sync with itself.

The browser already has a system for keeping things in sync. The cascade. It has been there since CSS2. It is highly optimized. It runs in C++ in the rendering engine. It uses incremental layout, dirty bits, GPU compositing. It is fast in ways JavaScript cannot match because it operates below JavaScript.

What this architecture names: applications spend most of their resources doing what the cascade already does, in JavaScript, slower, with worse architecture. Move the work to the cascade. The application runs on infrastructure that's already optimized for the work. Resource usage drops because the work isn't being duplicated in the wrong layer.

This is not "the solution to the resource usage epidemic" said in a way that overshoots. The defensible claim is more careful: for the class of applications that fit this architecture, JavaScript-as-write and render-as-read collapses an enormous amount of currently-duplicated work onto infrastructure the platform already optimizes. The resource savings should be significant. How significant, and across what application classes, is empirical work that follows from the substrate being verified at scale.

The class of applications that fit this architecture is bigger than it might first appear. Branch logic is sub-cascades. Sequencing is path geometry. Iteration is spiral structure. The procedural work that current architectures handle imperatively - branches, loops, control

flow, state mutation – is expressible as geometry the cascade resolves. The genuinely procedural residue is the moment external input arrives, which JavaScript-as-wire handles. That residue is small. The bulk of what applications do can move into the cascade if the substrate makes the move possible, and the substrate exists.

For real-time collaboration specifically: the architecture is structurally well-suited because collaborative state is inherently delta-shaped, and JavaScript has been forcing it into object-shaped representations that have to be re-invented as deltas at every transport boundary. The substrate makes the deltas first-class. The merge problem reduces to its true minimum residue – genuinely shared coordinates with application-specific resolution semantics – rather than appearing at every layer.

On what we can honestly say is over there

The discipline of this booklet is to describe the far side as what the specification commits to, not as a prediction about how widely it will be adopted or how much it will reshape the industry. The architecture is what it is. What it becomes is a separate question, downstream of work that is still ongoing.

What is known structurally:

- The substrate exists as specified primitive with verified empirical properties (228 passing tests at the time of writing this booklet, alignment validation against WASM ground truth at Spearman 0.85+, recursive feedback verified, persistence round-trip verified, browser deployment verified)
- The cascade is, in fact, a parallel resolver that the architecture can target
- The DOM is, in fact, a coordinate space that the architecture can claim
- CSS custom properties are, in fact, typed parametric state that the architecture can use
- WebGPU is, in fact, a substrate-equivalent execution surface for constraint resolution
- The deployment topology composes with SSR/CSR using existing browser infrastructure
- The delta-tension mechanism handles synchronization without a separate push channel
- Identity-scoped tree positions prevent most multi-user merge conflicts at the architecture level

What is not known empirically:

- Whether the substrate's configuration is rich enough that interactions on its projection produce application-equivalent behavior at the scale real applications operate at

- Whether the migration path - feeding existing source through the substrate - produces useful configurations across the diversity of source types real applications are written in
- Whether the resource savings, when measured against current production architectures, match the structural argument
- Whether the developer experience is good enough that the migration cost is repaid by the operational benefits
- Whether the standards and ecosystem work that would make this widely adoptable is achievable

These are the empirical questions that follow. The architecture is positioned to test them. The work that comes next is testing them.

On the sentence that started everything

Return to where this began.

Geometrically defining and mapping languages, operations, syntax, constraints, rules, standards each as their own separate multidimensional object, AND using uncertainty as a tie breaker, theoretically supports branch logical based computation to be traversed as trajectory in parallel vs executorial procedure.

Read it again with the architecture under it.

Geometrically defining and mapping languages, operations, syntax, constraints, rules, standards each as their own separate multidimensional object - this is the substrate's commitment that constraints are first-class. Languages are constraint sets. Operations are constraint matches. Syntax is a constraint over input shape. Rules are constraints. Standards are constraints. All of them are objects in the same grammar.

AND using uncertainty as a tie breaker - this is delta. The architecture's central measurement. The thing that decides which coordinate the field is closest to when multiple coordinates match. Uncertainty is not noise to be eliminated; uncertainty is the structural signal that lets the field move.

theoretically supports branch logical based computation - the cascade is a bounded decidable transition function. Branches are sub-cascades. Logic is structure. Computation is what the cascade resolves to.

to be traversed as trajectory in parallel - parallel by construction, because the substrate is shared and the cascade resolves the shared substrate without sequential coordination. Trajectory because operations leave traces, and the trace is what the field's history lives in.

vs executional procedure - against. As distinct from. As an alternative to. The sentence names what the architecture is not, in the same breath as what it is. Procedural execution is what it leaves behind. Trajectory through geometry is what it becomes.

The sentence held the whole arc. Two and a half years of architectural work has been walking out the consequences of what the sentence committed to. The substrate is the primitive that makes the trajectory traversable. The cascade is the resolver that runs in every browser already deployed. The DOM is the coordinate space the trajectory lives in. JavaScript is the wire that connects external interaction to substrate observation. The deployment topology composes over infrastructure the world has already built.

The far side of the canyon is real. The architecture that crosses to it exists in working code. The empirical work to verify it composes at the scale of real applications continues.

The sentence was right.

CONTROVERSIAL FOOTER

The web platform was built for documents and accidentally became infrastructure for a higher-dimensional state-space resolver, the cascade has been running this resolver in C++ for twenty-five years while the entire JavaScript ecosystem has been duplicating its work in a slower layer above it, and the substrate this booklet describes claims that infrastructure for its proper purpose by reducing JavaScript to a single function (write attribute on event), eliminating virtual DOM reconciliation entirely, replacing imperative state management with cascade-resolved coordinate-space, and collapsing the resource usage of typical web applications by an order of magnitude or more - not as optimization within the imperative paradigm but as the dimensionality expansion that makes the imperative paradigm a coordinate within the next one. Applications stop being closures with side effects and become structured artifacts whose state lives in CSS custom properties, whose structure lives in DOM tree position, whose behavior emerges from cascade resolution against substrate-shaped coordinates, whose multi-user composition uses identity-scoped tree positions and delta-shaped propagation, whose persistence is the default rather than a feature, whose resource cost is dominated by sunk infrastructure investment in a system already deployed at planetary scale. The von Neumann assumption that descriptions must be re-executed imperatively on every interaction was always hardware artifact masquerading as logical necessity; the brain solved this problem two billion years ago using twenty watts of geometric sampling against a static structure; the GPU pipeline in every device already deployed was built for exactly this class of operation; the architecture this booklet describes runs in any modern browser, holds 228 passing tests, validates at Spearman 0.85+ alignment with WASM ground truth, persists across sessions through

IndexedDB, and the only reason it isn't already the next platform is that it requires the same paradigm-shift recognition that virtualization required in 2012 and that nothing about the timing requires the recognition to take that long this time.

Supporting files: DEFINITION.md, INVARIANTS.md, KERNEL.md, MANIFEST.md, IMPLEMENTATION_PATH.md, PROJECT_SPLIT.md, AMENDMENT_2.4.md, ROADMAP.md, README.md, CONTRIBUTING.md, EXODUS spec.pdf, The_Vessel.text, SE-01-compositional-cascades.md, SE-02-metabolism.md, SE-03-field-modulation.md, SE-04-seed-constraint.md, SE-05-vector-delta-predictive-reaching.md, se-06.md, SE-07_Configuration-and-Settling.md, 00-INDEX.md, 01-manifold-reflex-primitives.md, 02-delta-computation.md, 03-delta-ipc-channel-fidelity.md, 04-constraint-compilation.md, 18-send-receive-potential-reference-tetrad.md, 19-observer-as-channel-triadic.md, 20-four-constraint-minimum-processor.md, PHASE_5_7_OBSERVATIONS.md, PHASE_5_7_5_OBSERVATIONS.md, PHASE_5_7_6_OBSERVATIONS.md.